

第五章 STM32L152 GPIO控制器

5.1 GPIO控制器的基本原理

5.2 GPIO控制器相关寄存器

5.3 GPIO控制器库函数

5.4 GPIO控制器使用示例



第五章 STM32L152 GPIO控制器

教学目标

- 掌握GPIO控制器的基本概念和组成
- 掌握GPIO 使用的基本流程
- 掌握常用GPIO库函数及其使用
- 掌握GPIO控制的程序实现

GPIO是嵌入式微控制器最常用、最基础、灵活性最强的控制器，可以实现简单控制、也可以组合实现复杂时序，是嵌入式程序设计必须掌握的控制器



5.1 GPIO控制器的基本原理

GPIO功能

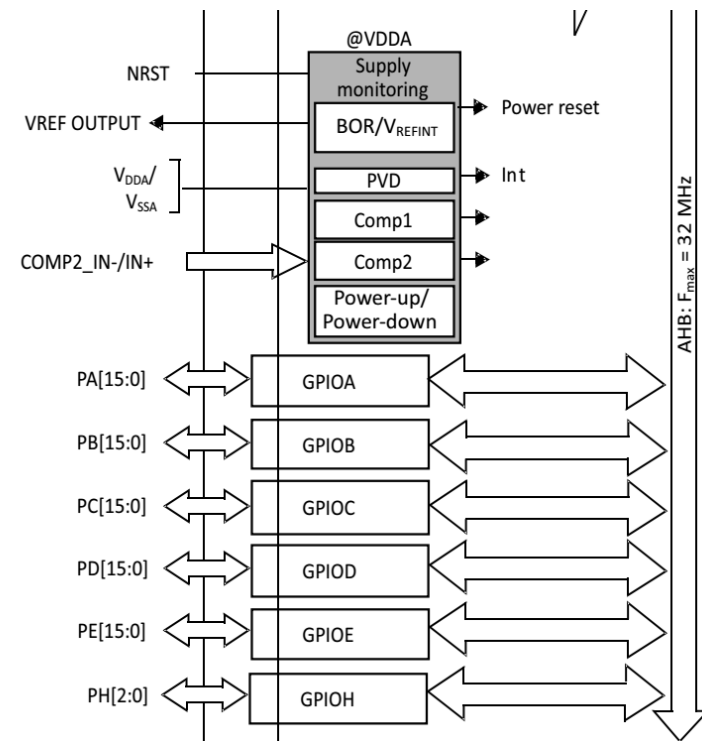
- **GPIO (General Purpose Input Output)**
- **MCU与外界进行信息交换，需要通过一些数据管脚实现数据的输入和输出，这些数据管脚或数据交换接口成为通用输入输出接口。**
- **当这些接口被用作USART、IIC等时，他们成为USART、IIC的专用接口。**
- **STM32L152系列微控制器可以提供多达80个通用IO端口（根据芯片的封装大小决定）**
- **实现这些IO配置和管理的外设：GPIO控制器。**



5.1 GPIO控制器的基本原理

GPIO功能

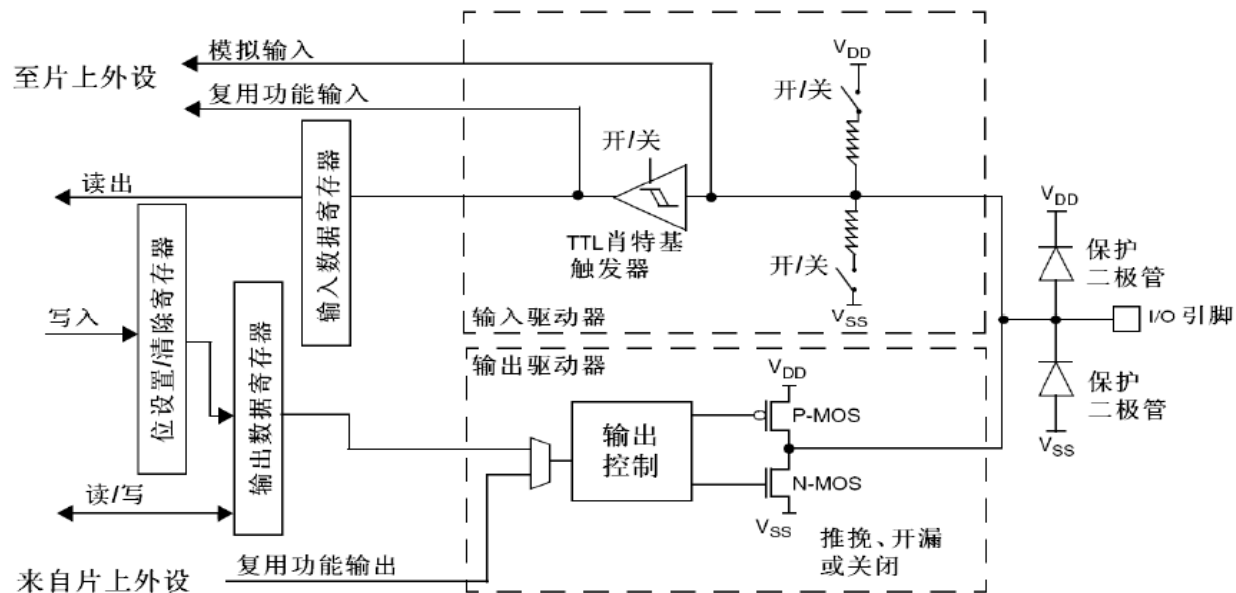
- STM32L1xx系列处理器将IO分为A~H 8组，每组称为一个端口（PORT），每个端口有16个IO管脚（PIN），支持不同的输入输出模式，每个IO的速度可单独配置，最快可在2个时钟周期进行IO翻转，支持IO锁定功能、IO复选功能，每个管脚最多可有16个复选功能，最大程度的提供了灵活的IO配置和使用。
- 最多有8个GPIO控制器
- STM32L152RBT6 64管脚，共有51个通用IO管脚



5.1 GPIO控制器的基本原理

GPIO内部结构

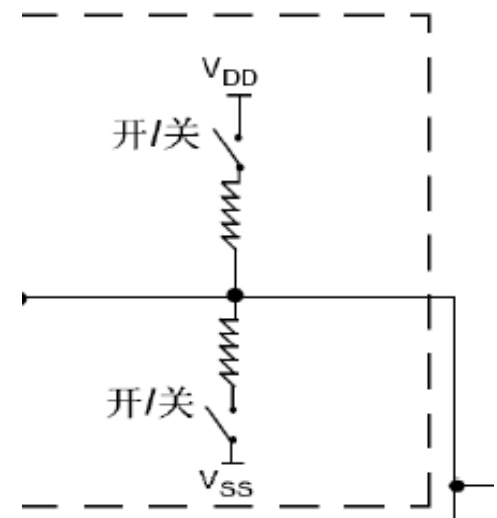
- 一个IO管脚的内部电路结构：
 - 输入驱动器、输出驱动器、输入数据寄存器、输出数据寄存器、位设置/清除寄存器



5.1 GPIO控制器的基本原理

GPIO内部结构

- GPIO端口的每个管脚可以由软件分别配置成多种模式：
 - 输入、输出、复用和模拟四种模式，由输入驱动器和输出驱动器的开关电路控制，每个管脚都可以单独配置和管理；
 - 端口配置为输出模式时，GPIO将输出寄存器中的值输出到对应的IO管脚上；
 - 在每个AHB总线时钟周期，GPIO控制器采样IO管脚的输入电平，并将输入数据存储在输入寄存器中。
 - 所有GPIO引脚有一个内部弱上拉和弱下拉，当配置为输入时，它们可以被激活也可以被断开。



5.1 GPIO控制器的基本原理

GPIO内部结构

- **GPIO控制器的特色：**
 - **GPIO控制器提供了单个或多个位的原子读写操作，通过对“置位/复位寄存器” (GPIOx_BSRR)中想要更改的位写1实现同时操作，无需软件进行开关中断的保护操作。**
 - **正常流程： 读取数据寄存器值→修改新的配置值→写入寄存器**
 - **提供了锁定机制允许冻结IO配置。当在一个端口位上执行了锁定(LOCK)程序，在下一次复位之前，将不能再更改端口位的配置。**
 - **所有的GPIO端口都有外部中断能力，将端口配置位输入模式后，可以把管脚作为中断源的输入**

5.1 GPIO控制器的基本原理

GPIO内部结构

- **GPIO控制器的特色：**
 - **所有的IO端口及其管脚都具有复用功能**
 - **将GPIO管脚映射到某个其它外围控制器上，作为专用IO通道；**
 - **每个IO管脚最多可以有16个复选功能AF0-AF15，但只能使用其中一个；**
 - **CPU上电复位后，所有的IO都默认使用AF0功能；**
 - **除了特殊IO管脚外，其它IO管脚的AF0功能都是GPIO；**
 - **IO口重映射：**
 - **把一些复用功能的管脚重新映射到其他一些管脚上。**

5.1 GPIO控制器的基本原理

GPIO模式配置

- 输入模式配置
 - 浮空输入
 - 上拉输入
 - 下拉输入
- 输出模式配置
 - 推挽输出
 - 开漏输出
- 复用功能配置
- 模拟配置



5.1 GPIO控制器的基本原理

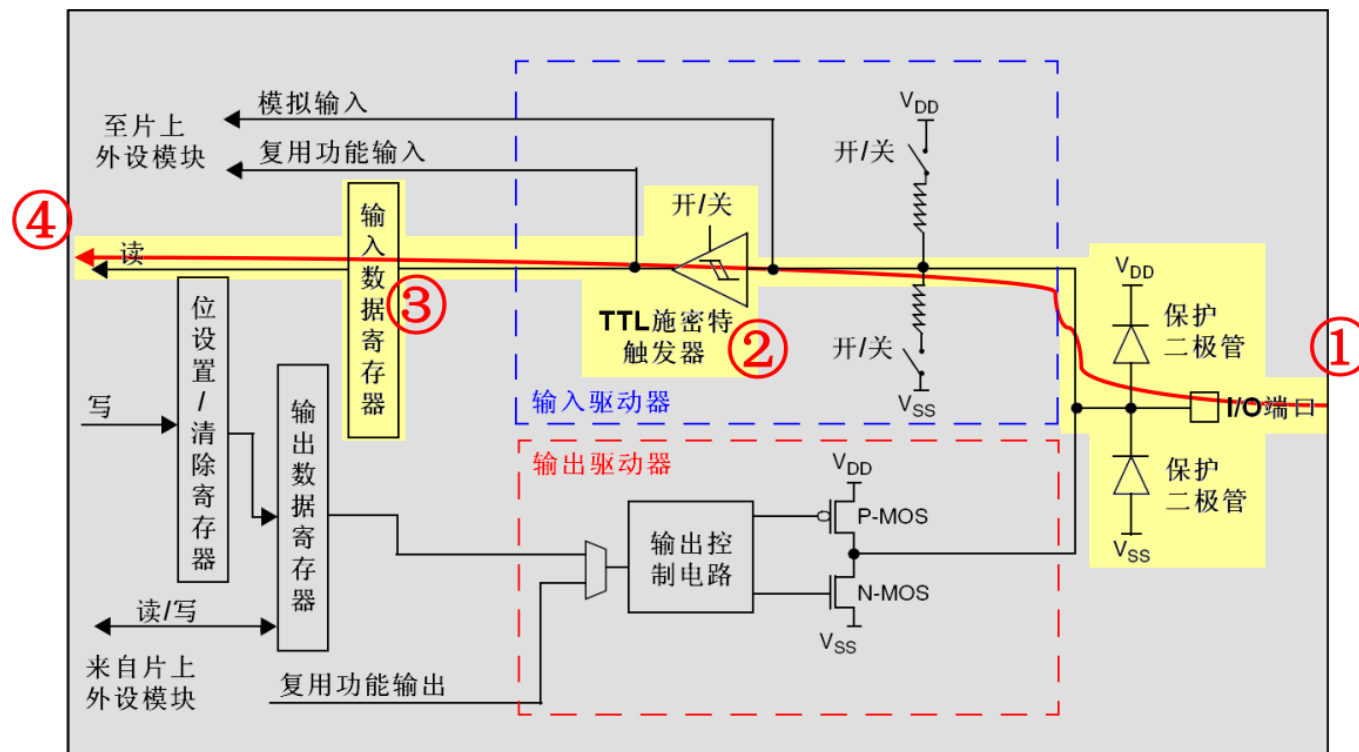
GPIO模式配置

- 输入模式配置
 - 输出控制被关闭，IO数据被采样到输入数据寄存器
 - 浮空输入：IO电平不确定，完全由外部输入信号决定
 - 上拉输入：上拉电阻连接，输入信号默认为高电平
 - 下拉输入：

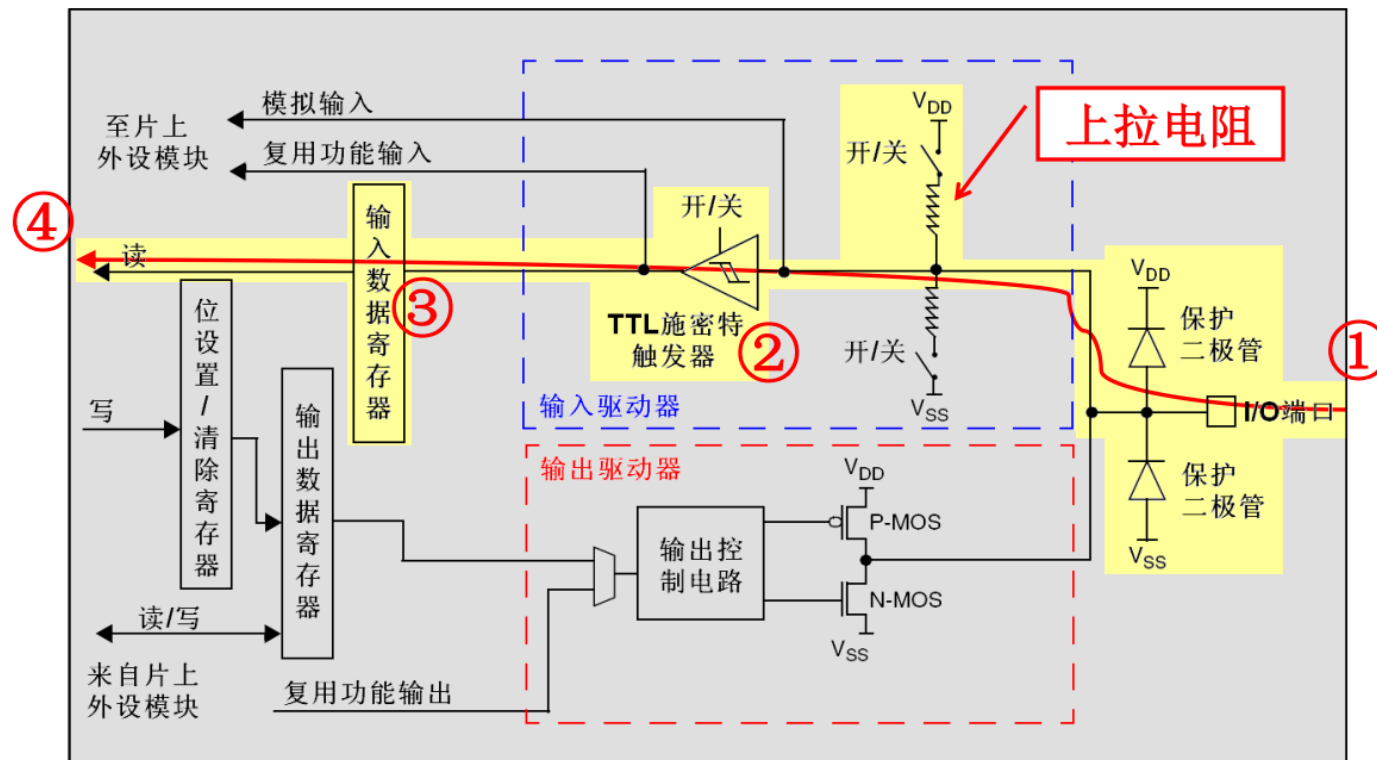
下拉电阻连接，
输入信号默认
为低电平

上拉、下拉、浮空
指的是没有信号输
入时的电平状态！！

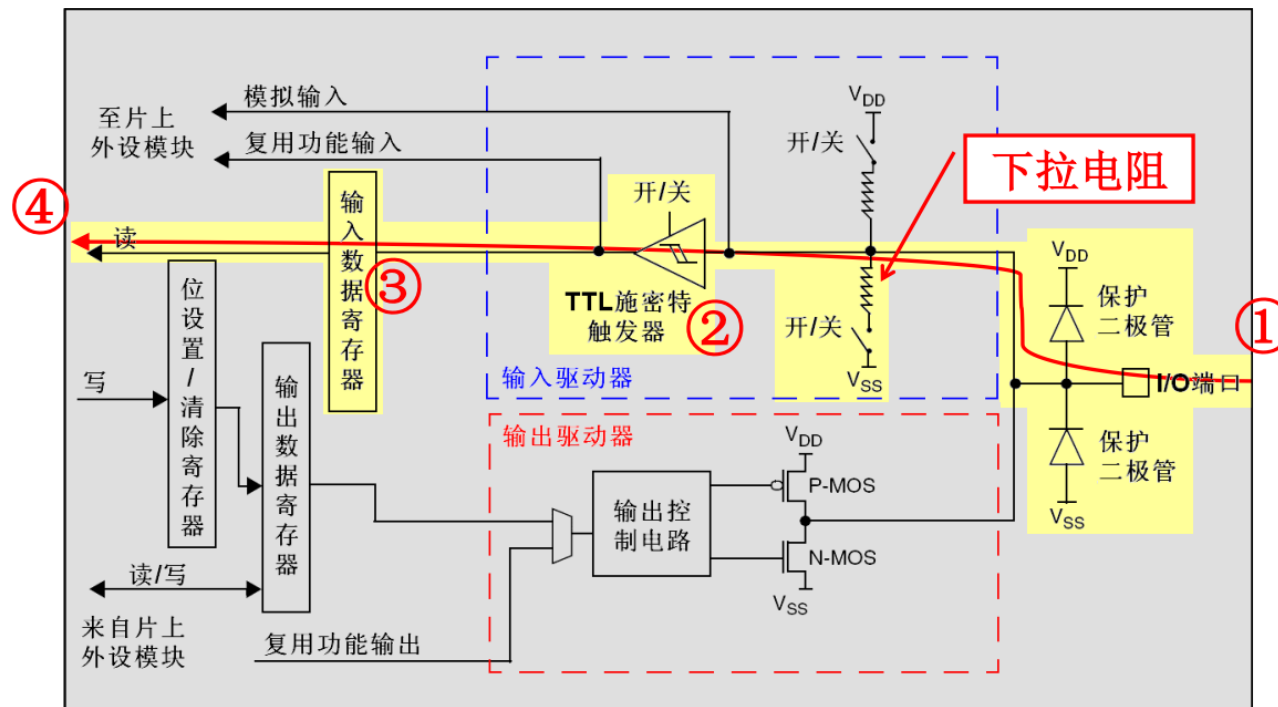
◆ GPIO的输入工作模式1—输入浮空模式



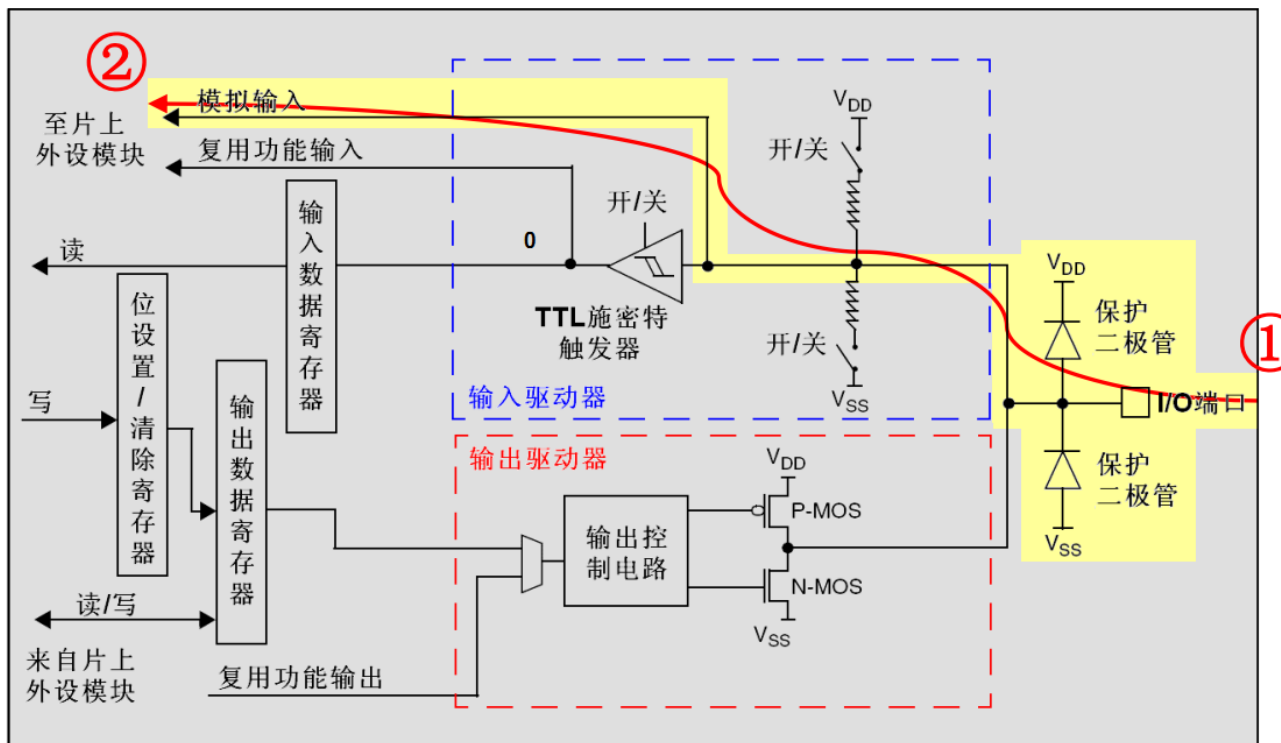
◆ GPIO的输入工作模式2—输入上拉模式



◆ GPIO的输入工作模式3—输入下拉模式



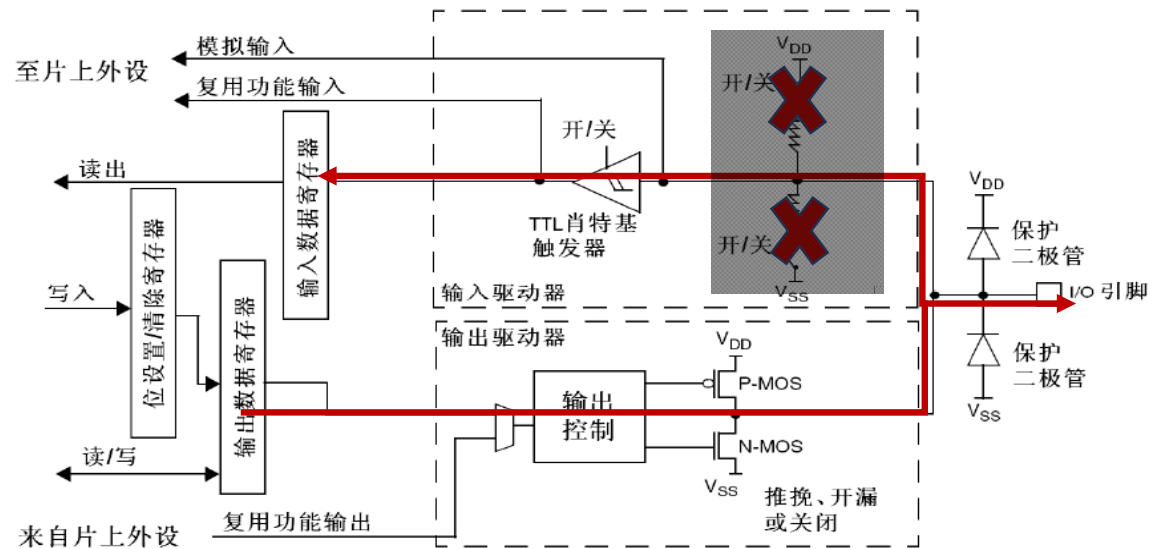
◆ GPIO的输入工作模式4—模拟模式



5.1 GPIO控制器的基本原理

GPIO模式配置

- 输出模式配置
- 上拉下拉关闭，数据从输出寄存器输出到IO管脚，同时IO管脚的电平被输入寄存器读入
 - 推挽输出
 - 开漏输出

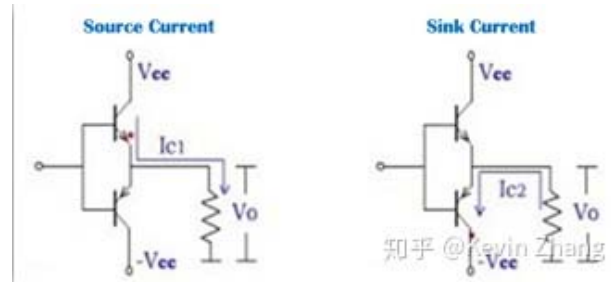
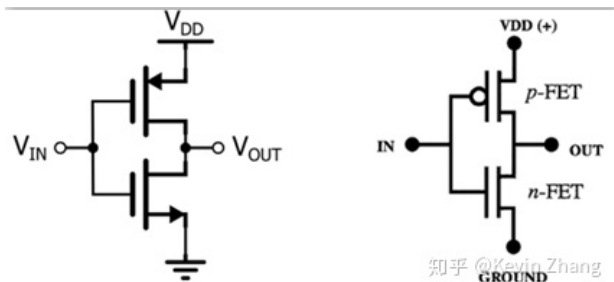
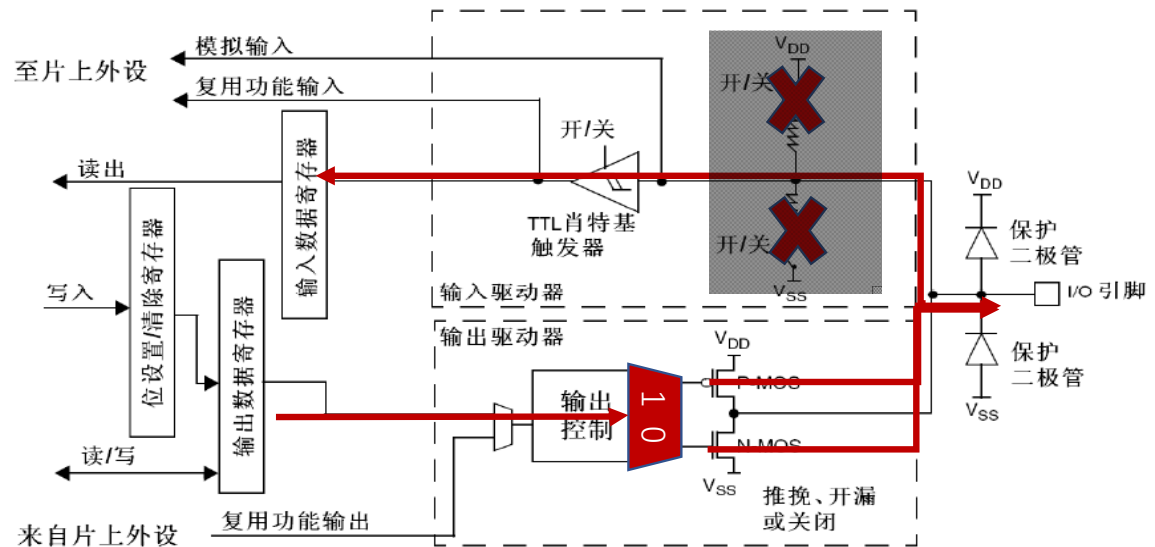


5.1 GPIO控制器的基本原理

GPIO模式配置

- 输出模式配置
- 推挽输出Push-Pull

推拉式电路，可以输出高电平、低电平，分别驱动PMOS和NMOS，效率高、负载能力强，速度快，输出电平的电压由VDD决定，一般用来连接数字器件

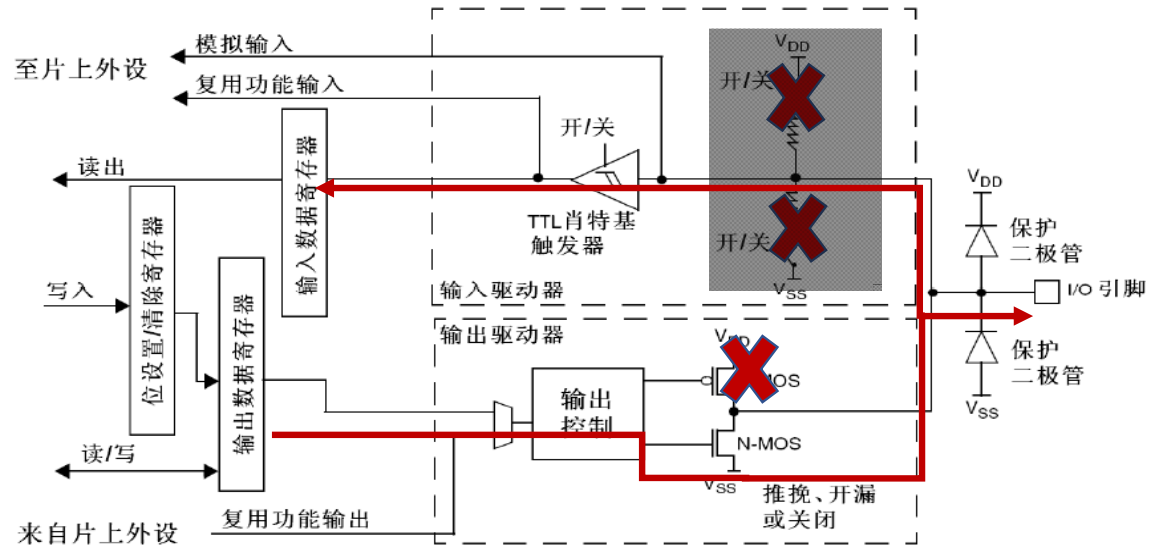


5.1 GPIO控制器的基本原理

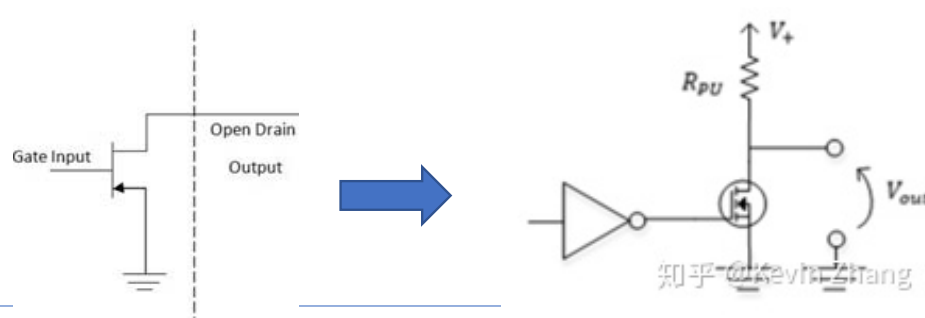
GPIO模式配置

- 输出模式配置
- 开漏输出Open-Drain

0驱动NMOS管，输出为0，
1驱动NMOS管，输出为高阻态，
只能输出0，不能输出1，需要上
拉电阻，响应速度与电阻的大小
有关



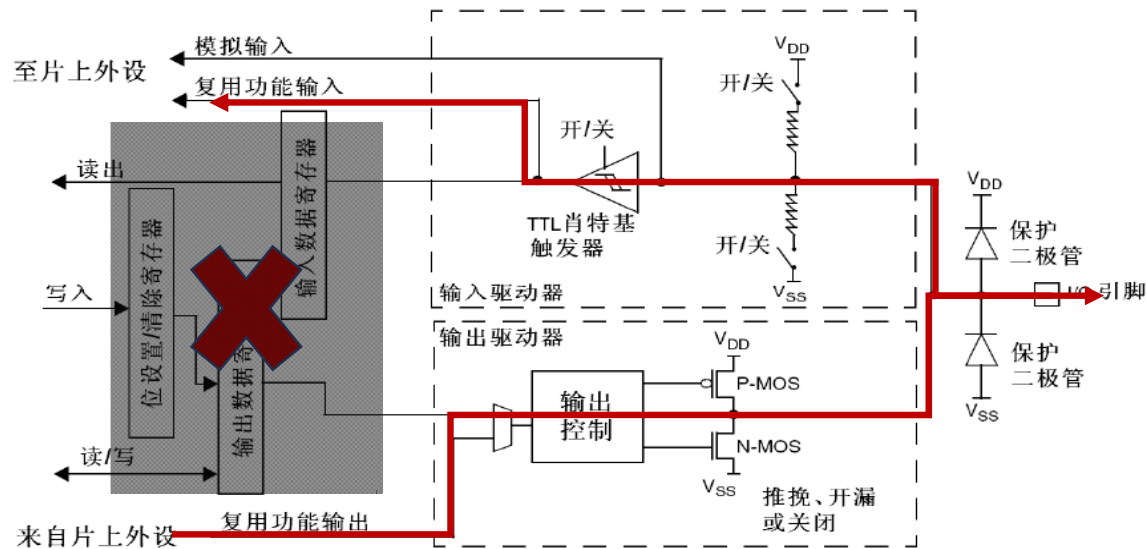
优点：吸收电流能力强,可以实现“线与”的功能



5.1 GPIO控制器的基本原理

GPIO模式配置

- 复用功能配置
 - IO管脚被配置为复用功能
 - 复用输出：开漏或推挽，输出缓冲器被打开，输出缓冲器由内置外设的信号驱动输出；
- 复用输入：施密特触发输入被激活，可以配置弱上拉和下拉电阻，数据送往内部外设控制器；
- 双向复用：配置为复用输出模式(推挽或开漏)，输入驱动器被置成浮空输入，和输出寄存器断开，与片上外设的输出信号连接。



5.1 GPIO控制器的基本原理

GPIO模式配置总结

应用场景	配置
普通GPIO输入	配置该引脚为浮空输入、带弱上拉输入或带弱下拉输入，不使能该引脚对应的所有复用功能模块
普通GPIO输出	根据需要配置该引脚为推挽输出或开漏输出，不使能该引脚对应的所有复用功能模块
普通模拟输入	配置该引脚为模拟输入模式，不使能该引脚对应的所有复用功能模块
内置外设的输入	根据需要配置该引脚为浮空输入、带弱上拉输入或带弱下拉输入，使能该引脚对应的复用功能模块
内置外设的输出	根据需要配置该引脚为复用推挽输出或复用开漏输出，使能该引脚对应的复用功能模块。



5.2 GPIO控制器相关寄存器

- 四个32位配置寄存器
 - 端口模式寄存器 GPIOx_MODER
 - 端口输出类型寄存器 GPIOx_OTYPER
 - 端口输出速度寄存器 GPIOx_OSPEEDR
 - 端口上拉下拉寄存器 GPIOx_PUPDR
- 两个32位数据寄存器
 - 输入数据寄存器 GPIOx_IDR
 - 输出数据寄存器 GPIOx_ODR
- 一个32位置位/复位寄存器
 - 端口位设置/清除寄存器 GPIOx_BSRR
- 一个32位锁定寄存器
 - 端口锁定寄存器 GPIOx_LCKR
- 两个32位的复选功能选择寄存器
 - 复选功能高寄存器 GPIOx_AFRH
 - 复选功能低寄存器 GPIOx_AFRL



5.3 GPIO控制器库函数

头文件: stm32f10x_gpio.h

源文件: stm32f10x_gpio.c

◆ 重要函数:

1个初始化函数:

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
```

2个读取输入电平函数:

```
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);  
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx);
```

2个读取输出电平函数:

```
uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);  
uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);
```

4个设置输出电平函数:

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);  
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);  
void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal);  
void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);
```



5.3 GPIO控制器库函数

- CMSIS中GPIO控制器的数据结构定义在stm32lxx.h中

```
typedef struct
{
    __IO uint32_t MODER; //模式寄存器, 地址偏移量: 0x00
    __IO uint16_t OTyPER; //输出类型寄存器, 地址偏移量: 0x04
    uint16_t RESERVED0; //保留, 4字节对齐用
    __IO uint32_t OSPEEDR; //输出速度寄存器, 地址偏移量: 0x08
    __IO uint32_t PUPDR; //上拉下拉寄存器, 地址偏移量: 0x0C
    __IO uint16_t IDR; //输入数据寄存器, 地址偏移量: 0x10
    uint16_t RESERVED1; //保留, 4字节对齐用
    __IO uint16_t ODR; //输出数据寄存器, 地址偏移量: 0x14
    uint16_t RESERVED2; //保留, 4字节对齐用
    __IO uint16_t BSRRL; //位设置/清除低寄存器, 地址偏移量: 0x18
    __IO uint16_t BSRRH; //位设置/清除高寄存器, 地址偏移量: 0x1A
    __IO uint32_t LCKR; //锁配置寄存器, 地址偏移量: 0x1C
    __IO uint32_t AFR[2]; //复用功能寄存器, 地址偏移量: 0x20-0x24
    __IO uint16_t BRR; //位清除寄存器, 地址偏移量: 0x28
    uint16_t RESERVED3; //保留, 4字节对齐
} GPIO_TypeDef;
```

```
#define __IO volatile
```

5.3 GPIO控制器库函数

- Stm32lxx.h中对8个GPIO控制器进行了声明

```

/*!< Peripheral memory map */
#define APB1PERIPH_BASE    PERIPH_BASE
#define APB2PERIPH_BASE    (PERIPH_BASE + 0x10000)
#define AHBPERIPH_BASE    (PERIPH_BASE + 0x20000)
#define GPIOA_BASE        (AHBPERIPH_BASE + 0x0000)
#define GPIOB_BASE        (AHBPERIPH_BASE + 0x0400)
#define GPIOC_BASE        (AHBPERIPH_BASE + 0x0800)
#define GPIOD_BASE        (AHBPERIPH_BASE + 0x0C00)
#define GPIOE_BASE        (AHBPERIPH_BASE + 0x1000)
#define GPIOH_BASE        (AHBPERIPH_BASE + 0x1400)
#define GPIOF_BASE        (AHBPERIPH_BASE + 0x1800)
#define GPIOG_BASE        (AHBPERIPH_BASE + 0x1C00)
#define GPIOA              ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB              ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC              ((GPIO_TypeDef *) GPIOC_BASE)
#define GPIOD              ((GPIO_TypeDef *) GPIOD_BASE)
#define GPIOE              ((GPIO_TypeDef *) GPIOE_BASE)
#define GPIOH              ((GPIO_TypeDef *) GPIOH_BASE)
#define GPIOF              ((GPIO_TypeDef *) GPIOF_BASE)
#define GPIOG              ((GPIO_TypeDef *) GPIOG_BASE)

```

GPIOA是一个指针

GPIOA->MODER = 0x00000001

5.3 GPIO控制器库函数

- ST提供了GPIO标准库函数，头文件为stm32l1xx_gpio.h，程序源代码为stm32l1xx_gpio.c

GPIO_InitTypeDef结构体用于IO口的初始化配置

typedef struct

```
{
    uint32_t GPIO_Pin; // 需要被配置的IO管脚
    GPIO_Mode_TypeDef GPIO_Mode; // IO模式，用于对模式寄存器配置
    GPIO_Speed_TypeDef GPIO_Speed; // IO速率，用于对速率寄存器配置
    GPIO_OType_TypeDef GPIO_OType; // 输出管脚类型，用于配置输出类型寄存器
    GPIO_PuPd_TypeDef GPIO_PuPd; // 上拉下拉类型，用于配置上拉下拉寄存器
}GPIO_InitTypeDef;
```

typedef enum

```
{
    GPIO_Mode_IN = 0x00, // 输入模式
    GPIO_Mode_OUT = 0x01, // 输出模式
    GPIO_Mode_AF = 0x02, // 复用功能
    GPIO_Mode_AN = 0x03, // 模拟模式
}GPIO_Mode_TypeDef;
```

typedef enum

```
{
    GPIO_PuPd_NOPULL = 0x00, // 浮空
    GPIO_PuPd_UP = 0x01, // 上拉
    GPIO_PuPd_DOWN = 0x02, // 下拉
}GPIO_PuPd_TypeDef;
```

typedef enum

```
{
    GPIO_OType_PP = 0x00, // 推挽输出
    GPIO_OType_OD = 0x01, // 开漏输出
}GPIO_OType_TypeDef;
```

typedef enum

```
{
    GPIO_Speed_400KHz = 0x00, // 极低速
    GPIO_Speed_2MHz = 0x01, // 低速
    GPIO_Speed_10MHz = 0x02, // 中等速度
    GPIO_Speed_40MHz = 0x03, // 高速
}GPIO_Speed_TypeDef;
```

GPIO_Pin的取值定义如下:

```
#define GPIO_Pin_0 ((uint16_t)0x0001)
#define GPIO_Pin_1 ((uint16_t)0x0002)
#define GPIO_Pin_2 ((uint16_t)0x0004)
#define GPIO_Pin_3 ((uint16_t)0x0008)
#define GPIO_Pin_4 ((uint16_t)0x0010)
#define GPIO_Pin_5 ((uint16_t)0x0020)
#define GPIO_Pin_6 ((uint16_t)0x0040)
#define GPIO_Pin_7 ((uint16_t)0x0080)
#define GPIO_Pin_8 ((uint16_t)0x0100)
#define GPIO_Pin_9 ((uint16_t)0x0200)
#define GPIO_Pin_10 ((uint16_t)0x0400)
#define GPIO_Pin_11 ((uint16_t)0x0800)
#define GPIO_Pin_12 ((uint16_t)0x1000)
#define GPIO_Pin_13 ((uint16_t)0x2000)
#define GPIO_Pin_14 ((uint16_t)0x4000)
#define GPIO_Pin_15 ((uint16_t)0x8000)
#define GPIO_Pin_All ((uint16_t)0xFFFF)
```

5.3 GPIO控制器库函数

- ST提供了GPIO标准库函数，头文件位stm32l1xx_gpio.h，程序源代码位stm32l1xx_gpio.c

函数名	描述
GPIO_DeInit	将外设 GPIOx 寄存器重设为缺省值
GPIO_Init	根据 GPIO_InitStruct 中指定的参数初始化外设 GPIOx 寄存器
GPIO_StructInit	把 GPIO_InitStruct 中的每一个参数按缺省值填入
GPIO_ReadInputDataBit	读取指定端口管脚的输入
GPIO_ReadInputData	读取指定的 GPIO 端口输入
GPIO_ReadOutputDataBit	读取指定端口管脚的输出
GPIO_ReadOutputData	读取指定的 GPIO 端口输出
GPIO_SetBits	设置指定的数据端口位
GPIO_ResetBits	清除指定的数据端口位
GPIO_WriteBit	设置或者清除指定的数据端口位
GPIO_Write	向指定 GPIO 数据端口写入数据
GPIO_PinLockConfig	锁定 GPIO 管脚设置寄存器
GPIO_PinAFConfig	选择 GPIO 管脚的复用功能



5.3 GPIO控制器库函数

• GPIO_Init函数

函数功能：根据参数GPIO_InitStruct 初始化GPIOx的寄存器

函数原型：void GPIO_Init(GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_InitStruct)。

输入参数：

GPIOx：需要配置的IO端口，取值为GPIOA、GPIOB... GPIOH。

GPIO_InitStruct：指向已初始化成员的GPIO_InitTypeDef 结构体变量的指针。

应用示例：

GPIO_InitTypeDef GPIO_InitStructure;

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5; //LED0-->PB.5 端口配置

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出

GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //IO口速度为50MHz

GPIO_Init(GPIOB, &GPIO_InitStructure); //根据设定参数初始化GPIOB.5

5.3 GPIO控制器库函数

◆ 2个读取输出电平函数:

uint8_t GPIO_ReadOutputDataBit (GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

作用: 读取某个**GPIO**的输出电平。实际操作的是**GPIO_ODR**寄存器。

例如:

```
GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_5); //读取GPIOA.5的输出电平
```

uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);

作用: 读取某组**GPIO**的输出电平。实际操作的是**GPIO_ODR**寄存器。

例如:

```
GPIO_ReadOutputData(GPIOA); //读取GPIOA组中所有io口输出电平
```

5.3 GPIO控制器库函数

◆ 2个读取输出电平函数：

uint8_t GPIO_ReadOutputDataBit (GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

作用：读取某个**GPIO**的输出电平。实际操作的是**GPIO_ODR**寄存器。

例如：

```
GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_5); //读取GPIOA.5的输出电平
```

uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);

作用：读取某组**GPIO**的输出电平。实际操作的是**GPIO_ODR**寄存器。

例如：

```
GPIO_ReadOutputData(GPIOA); //读取GPIOA组中所有io口输出电平
```

5.3 GPIO控制器库函数

• GPIO_ResetBits函数

函数功能：清除指定IO端口的指定IO管脚数据

函数原型：void GPIO_ResetBits (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)

输入参数：

- GPIOx：所要清除的IO端口，取值为GPIOA、GPIOB ... GPIOH。
- GPIO_Pin：所要清除的IO管脚，取值为GPIO_Pin_x, x=0~15，可以是多个管脚，GPIO_Pin的每位表示一个管脚号。

应用示例：

```
GPIO_ResetBits(GPIOA, GPIO_Pin_10 | GPIO_Pin_15);
```

5.3 GPIO控制器库函数

• GPIO_SetBits函数

函数功能：设置指定IO端口的指定IO管脚数据

函数原型：void GPIO_SetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)

输入参数：

- GPIOx：所要设置的IO端口，取值为GPIOA、GPIOB ... GPIOH。
- GPIO_Pin：所要设置的IO管脚，取值为GPIO_Pin_x, x=0~15, 可以是多个管脚，GPIO_Pin的每位表示一个管脚号。

应用示例：

```
GPIO_SetBits(GPIOA, GPIO_Pin_10 | GPIO_Pin_15);
```



5.3 GPIO控制器库函数

• GPIO_ToggleBits函数

函数功能：将指定端口的IO管脚数据翻转。

函数原型：void GPIO_ToggleBits(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)。

输入参数：

- GPIOx：所指定的IO端口，取值为GPIOA、GPIOB ... GPIOH。
- GPIO_Pin：指定的IO管脚，取值为GPIO_Pin_x, x=0~15。

应用示例：

```
GPIO_ToggleBits(GPIOA, GPIO_Pin_5 );
```

5.3 GPIO控制器库函数

• GPIO_Write函数

函数功能：为端口数据寄存器写入指定值。

函数原型：void GPIO_Write(GPIO_TypeDef * GPIOx, uint16_t PortVal)。

输入参数：

- GPIOx：需要写入的指定端口，取值为GPIOA、GPIOB ... GPIOH。
- PortVal：需要写入到端口数据寄存器的值

应用示例：

```
GPIO_Write(GPIOA, 0x1101);
```

5.3 GPIO控制器库函数

• GPIO_PinAFConfig函数

函数功能：设置IO端口的复选功能。

函数原型：void GPIO_PinAFConfig(GPIO_TypeDef * GPIOx, uint16_t GPIO_PinSource, uint8_t GPIO_AF)。

输入参数：

- GPIOx：需要修改的IO端口，取值为GPIOA、GPIOB... GPIOH。
- GPIO_PinSource：端口的IO管脚，取值为GPIO_PinSourcex, x=0~15，其定义如下：

```
#define GPIO_PinSource0      ((uint8_t)0x00)
```

```
#define GPIO_PinSource1      ((uint8_t)0x01)
```

```
#define GPIO_PinSource2      ((uint8_t)0x02)
```

GPIO_AFSelection：复选功能选择，其取值范围为：

```
//AF 0 selection
```

```
#define GPIO_AF_RTC_50Hz     ((uint8_t)0x00) //RTC 50/60 Hz
```

```
#define GPIO_AF_MCO          ((uint8_t)0x00) //MCO
```

```
#define GPIO_AF_RTC_AF1      ((uint8_t)0x00) //RTC_AF1
```

```
#define GPIO_AF_WKUP         ((uint8_t)0x00) //Wakeup (WKUP1, WKUP2, WKUP3)
```

```
#define GPIO_AF_SWJ          ((uint8_t)0x00) //SW、JTAG
```



5.4 GPIO控制器使用示例

GPIO寄存器基本操作示例

例1：读输入寄存器的寄存器实现。

```
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)
{
    //直接返回IDR寄存器的值
    return ((uint16_t)GPIOx->IDR);
}
```

5.4 GPIO控制器使用示例

GPIO寄存器基本操作示例

例2：配置一个引脚的输出值的寄存器实现。

```
void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)
{ //Bit_RESET =0
  if (BitVal != Bit_RESET)
  {
    //直接操作BSRRL将输出数据寄存器对应位清0
    GPIOx->BSRRL = GPIO_Pin;
  }
  else
  {
    //通过BSRRH将输出数据寄存器对应位置1
    GPIOx->BSRRH = GPIO_Pin ;
  }
}
```

5.4 GPIO控制器使用示例

LED控制

例：在STM32L152-DISCOVERY开发板上，通过连接到LED的GPIO管脚对绿色LED3灯实现反转控制。LED3的控制管脚为PB7，因此我们需要对GPIOB端口的第7个管脚进行配置，根据电路图，PB7输出为1时灯亮，输出0时灯灭。

```
/* Includes -----*/
#include "stm32l1xx.h"
#define GREEN_LED GPIO_Pin_7
#define BSRR_VAL 0x80
GPIO_InitTypeDef    GPIO_InitStructure;

void Delay(__IO uint32_t nCount)
{
    while(nCount--);
}
```

定义两个宏 GREEN_LED和BSRR_VAL

- GPIO_Pin_7为系统定义的宏变量，其值为0x07;
- BSRR_VAL 用于对BSRR寄存器进行赋值，BSRR_VAL的值设为0x80

GPIO_InitTypeDef 结构体类型定义一个GPIO寄存器结构体变量 GPIO_InitStructure，用于对GPIOB进行初始化

5.4 GPIO控制器使用示例

```
int main(void)
{
    /* GPIOD Periph clock enable */
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_GPIOD, ENABLE);
    /* Configure PB7 in output pushpull mode */
    GPIO_InitStructure.GPIO_Pin = GREEN_LED;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_40MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    while (1)
    {
        /* Set PB7 */
        GPIOD->BSRRL = BSRR_VAL;
        Delay(1000);
        /* Reset PB7 */
        GPIOD->BSRRH = BSRR_VAL;
        Delay(1000);
    }
}
```

调用RCC_AHBPeriphClockCmd使能GPIOB的时钟，此时GPIOB可以正常工作。

- 通过对BSRRL寄存器的第7位写1将GPIOB的输出数据寄存器第7位置1，灯亮
- 通过对BSRRH寄存器的第7位写1将GPIOB的数据寄存器第7位置0，灯灭

5.4 GPIO控制器使用示例

按键输入

例：通过开发板按键作为输入，当按键按下时，获取按键的值，控制LED3变亮，按键弹起时，控制LED3变灭。

根据STM32L152-DISCOVERY开发板的原理图，按键连接在PA0口上，当按键按下时，PA0为低电平，弹起时为高电平。本例程中需要使用两个IO口，PA0作为输入，PB7作为输出，主程序如下：

```
int main(void)
{
    GPIO_Configuration();
    while (1)
    {
        if(GPIO_ReadBits(GPIOA,GPIO_Pin_0))
            GPIO_SetBits(GPIOB, GPIO_Pin_7);
        else
            GPIO_ResetBits(GPIOB,GPIO_Pin7)
    }
}
```



5.4 GPIO控制器使用示例

```
void GPIO_Configuration(void)
{
    //配置LED3
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_40MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOB, &GPIO_InitStructure);
    //配置按键
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
    //使能或者失能 APB2 外设时钟
    RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOA|
    RCC_APB2Periph_GPIOB, ENABLE);
}
```

